

13. Education Section — Bilingual Sign-Language Learning

Purpose: This section documents the Education tab of the application at the same technical depth as the Translation & Chat features described earlier in this chapter: its three learning tools (Sign Viewer, Sentence Builder, and Practice mode), the motion-capture skeleton rendering that replaced the earlier hand-drawn stickman, the server-side animation stitching pipeline, the bilingual ASL/ArSL architecture, and the content verification methodology that guarantees every sign the page teaches is a genuine sign performed by a real signer.

The Education tab turns the application from a translation tool into a learning platform: instead of only translating a deaf user's signing for a hearing conversation partner, it teaches the hearing user to sign back. It supports both of the application's languages — American Sign Language (ASL, paired with English) and Arabic Sign Language (ArSL, paired with Arabic) — with the same three tools in each.

13.1 Overview and Layout

Purpose: To describe how the Education tab is composed and navigated.

The Education tab (`frontend/src/pages/EducationTab.jsx`) is one of the primary tabs on the App Home Page. A header sub-navigation offers a language toggle — “ASL · English” / “ArSL · العربية” — and the tab body presents three tools:

- **Sign Viewer:** browse a categorized vocabulary of word signs and watch each one performed by a motion-capture skeleton, or open the Fingerspelling view to spell any typed word letter by letter (Section 13.4).
- **Sentence Builder:** type any free-form sentence and watch the skeleton sign it end-to-end — dictionary words are signed, unknown words are automatically fingerspelled (Section 13.5).
- **Practice mode:** perform a prompted letter or word into the webcam and get instant correct/incorrect feedback from the same recognition models that power the live Chatting tab (Section 13.7).

Switching languages remounts the three tool components with a React `key={lang}`, so selection, playback and camera state reset naturally without manual state-synchronization logic. Each component receives its language-specific configuration (word list, categories, alphabet, text direction, placeholder copy) from a single `LANG_CONFIG` lookup, so the English and Arabic experiences share one code path.

13.2 Skeleton Rendering: Motion-Captured Signs

Purpose: To record why the education content was rebuilt on real motion-capture data and how it is rendered.

An earlier iteration animated signs with a hand-authored 2D canvas stickman: every sign was a manually written sequence of wrist keyframes and procedurally constructed handshapes. That approach could not scale in either quantity or quality — each new sign required careful manual authoring and geometric verification, and even a correct keyframe animation reads as a symbolic illustration rather than as a human performing the sign. The section was therefore rebuilt on skeleton recordings of real signers:

- **Data format:** .pose files — the standard binary format of the pose-format library — containing per-frame MediaPipe Holistic landmarks (body, face and both hands) extracted from video of a human signer.
- **Renderer:** the <pose-viewer> web component from the sign-language-processing project (the same renderer used by the open-source sign.mt translation platform), which plays a .pose file as an animated full-body skeleton with per-finger articulation. The component is driven imperatively from React: playback rate, play/pause, and completion (ended\$) and load (loadeddata\$) events.
- **Content source:** sign.mt's public dictionary API (fetch_signmt_pose.py), which translates a spoken-language word into a signed-language pose clip drawn from a curated dictionary — American Sign Language (ase) for English and Jordanian Sign Language (jos) for Arabic, the only openly available Arabic-lookup sign dictionary. Every clip is attributed on-page (“Motion captured from a real signer”).

One rendering subtlety is worth recording: the pose-viewer custom element upgrades lazily, and a false JSX boolean attribute is simply removed from the DOM — so `autoplay={false}` never reaches the component, whose internal default is true. Each component therefore sets `el.autoplay = false` imperatively in a mount effect. The legacy canvas stickman remains only as a fallback renderer for the few English words that have hand-authored keyframes but no .pose file.

13.3 Server-Side Animation Stitching (pose_concat.py)

Purpose: To describe how multiple sign clips are combined into one smooth, natural-looking animation instead of a choppy clip-after-clip playback.

Playing letter or word clips back-to-back on the client produced a visibly laggy experience: between every two clips the skeleton snapped from one pose to another, with a network fetch in between. All multi-sign playback was therefore moved server-side. The model-services FastAPI application stitches the requested signs into ONE .pose file using spoken-to-signed — sign.mt's own concatenation package — and streams the finished animation to the client:

1. Each clip is reduced (redundant landmarks dropped), normalized, and trimmed at natural boundaries (the wrist-above-elbow heuristic finds where signing actually starts and ends).
2. Adjacent clips are cut at their best connection point — the frame pair where the two skeletons are geometrically closest — so one sign flows into the next.
3. A short zero-confidence gap (0.2 s) is inserted at every boundary and filled by interpolation, then the whole sequence is smoothed with a Savitzky-Golay filter, wrist positions are corrected, and the result is size-normalized.

Three endpoints expose this pipeline, all language-parameterized:

- **/api/pose-seq (mode=words):** stitches a list of named signs with natural trimmed motion — used when the Sign Viewer plays a single word.
- **/api/pose-seq (mode=spell):** fingerspelling mode. Letter clips are inconsistent upstream (some are 3-frame stills, some full 25/30 fps clips), and connection-point cutting can shrink a letter to a few unreadable frames — so spell mode instead slices each letter to its central held handshape and freezes it for 0.35 s, producing a uniform, readable spelling rhythm.
- **/api/pose-text:** free-text mode for the Sentence Builder (Section 13.5).

Every stitching endpoint has a /meta twin returning per-segment durations on the final timeline, which the frontend uses to highlight the currently playing word chip or letter with plain timers — the animation itself is a single opaque file, so synchronized UI feedback must come from metadata. Results are memoized server-side (functools.lru_cache), and Content-Disposition headers use RFC 5987 encoding because Arabic filenames cannot be represented in latin-1 HTTP headers.

13.4 Sign Viewer

Purpose: To describe the vocabulary browser and fingerspelling tool (ASLWordStickman.jsx).

The Sign Viewer presents the language's vocabulary as category tabs (e.g. Everyday Words, People, Feelings; مشاعر وصفات، الناس، كلمات يومية، ...) with one button per word. Selecting a word previews its first frame; Play streams the sign at a chosen speed (Slow 0.5× or Normal 1×, the default). Each word is captioned with its meaning — for Arabic, an English gloss, since the app's UI language is English. A final tab, Fingerspelling, replaces the word grid with:

- **A letter reference grid** (A–Z for ASL; the 28 Arabic base letters for ArSL): tapping a letter plays its held handshape clip.
- **A free spelling input:** any typed word is filtered to spellable letters and played as one server-stitched spell-mode sequence, with the on-screen letter grid highlighting each letter in sync via the /meta timing data.

The English vocabulary counts 44 words (38 with motion-capture poses, plus grammar words for sentence building) and 26 letters; the Arabic vocabulary counts 22 browsable word signs across five categories plus 28 letters. If the configured default word is ever pruned from the vocabulary, the component falls back to the first available word rather than rendering an empty viewer.

13.5 Sentence Builder — Free-Text Signing

Purpose: To describe how an arbitrary typed sentence becomes one continuous signed animation (SentenceBuilder.jsx).

The Sentence Builder follows sign.mt's interaction model: the user types any sentence; words found in the sign dictionary are signed, and unknown words are fingerspelled letter by letter. The server's plan builder (pose_concat.build_text_sequence) implements this as follows:

1. The text is tokenized (lowercased; Arabic diacritics stripped; digits 1–5 and Arabic-Indic ٠–٩ mapped to their number words).
2. Tokens are matched greedily against the dictionary, longest phrase first (3-gram, then 2-gram, then single word) — multi-word entries such as “thank you” or “مع السلامة” exist as ONE pose file and must not be split into unknown singles and spelled.
3. Lookups are orthography-tolerant (Section 13.6): both the query and the stored filenames are folded before comparison.
4. Any token still unmatched is decomposed into its letters and fingerspelled inline, using the same held-peak spelling treatment as the Fingerspelling tab.
5. The full plan is stitched into one smooth animation and returned with per-token and per-letter timing metadata.

The frontend mirrors the same greedy matching client-side to render a live chip preview as the user types: solid chips are words that will be signed from the dictionary; dashed chips marked  will be fingerspelled. This honesty matters — a learner must never mistake a spelled word for the word's actual sign. During playback the chips and, for spelled tokens, the individual letters light up in sync with the animation. Input is bounded (12 tokens, 80 stitched segments) so a pathological sentence cannot exhaust the stitching service.

13.6 Arabic Support (ArSL)

Purpose: To document the Arabic-specific architecture: content, orthography handling, and known linguistic caveats.

Every pose endpoint takes `lang=en|ar` and resolves content from a per-language directory (`poses/` vs `poses/ar/`). The frontend components are fully parameterized: right-to-left text direction on Arabic inputs and labels, Arabic-Indic digit handling, and Arabic category labels. Two Arabic-specific engineering problems are worth recording:

- **Orthographic variance.** Arabic words are commonly typed without hamza/madda marks (أم for أم, آسف for آسف), yet the sign dictionary keys many entries under proper orthography — and vice versa. A single folding table (أ, إ, ئ, ي, و, ه, ء dropped) is applied to BOTH sides of every lookup: the server resolves queries through a folded-name index (`pose_concat.resolve_pose_name`), and the client mirrors the same fold for chip previews (`poseViewer.foldWord`) and practice-answer matching. Files keep their proper spellings; users may type either form.
- **Fingerspelling alphabet.** ArSL fingerspelling uses the 28 base letters; variant forms fold onto the letters that have clips (e.g. `ه` spells as `ه`), and diacritics are stripped before spelling.

Two linguistic caveats are documented for the team. First, the Arabic sign content is Jordanian Sign Language (`jos`) — the only openly available Arabic sign dictionary — while the ArSL recognition models

were trained on KArSL (Saudi) data; regional sign variants differ, so the viewer and the recognizer are not guaranteed to agree on every word. Second, sentence signing follows the typed word order (a transliteration); it does not reorder into natural sign-language grammar. Both are acceptable for a learning aid but should be stated honestly in any evaluation.

13.7 Practice Mode — Learn by Doing

Purpose: To describe the webcam practice loop and how it reuses the existing recognition pipeline (PracticePanel.jsx).

Practice mode closes the learning loop: after watching a sign, the user performs it. The panel reuses the exact camera-and-predict pipeline that powers the live Chatting tab — extracted into a shared useCamera hook — with its own camera instance so the two tabs never interfere:

- **Letter practice:** the panel prompts a random letter; the user poses it and a single webcam frame is sent to POST /predict (mode=letters). The models return exactly the comparison target — an uppercase English letter or a real Arabic glyph — so matching is a normalized string comparison (isLetterMatch).
- **Word practice:** the panel prompts a word, runs a 3-2-1 countdown (cued performance needs reaction time), records a 3.2-second clip, and sends it to POST /predict (mode=words). Arabic matching folds orthography on both sides so the model's bare-spelled labels (أنا) match the vocabulary's proper spellings (أنا).
- **Honest target lists:** only words the deployed checkpoints can actually recognize are offered for checking — the 12 education words inside the WLASL-100 class list for ASL, and the 6-word fold-normalized intersection with the KArSL model's 32 labels for ArSL (لا، اسم، مع السلامة، أنا، آسف، دقيقة). Everything else is watch-only rather than silently impossible.
- **Constructive feedback:** when the model saw a sign but stayed below its confidence threshold, the UI reports its best guess (“it looked like X, just not clearly enough”) instead of a generic failure, distinguishing “almost had it” from “wrong sign entirely”.

Serving the legacy recognition models required its own engineering: the checkpoints were trained on Keras 2.10, which cannot run on the service's Python 3.12. A compatibility loader (legacy_h5.py) rewrites each .h5 file's serialized config — removing arguments Keras 3 no longer accepts — and loads the result through Keras 3 on a PyTorch backend, preserving the trained weights unchanged.

13.8 Content Verification — Defeating the Silent Fallback

Purpose: To document the methodology that guarantees every vocabulary entry is a genuine sign, which proved to be the hardest data-quality problem in the section.

The sign.mt dictionary API has a critical property: for a word its dictionary does not contain, it silently returns a FINGERSPELLED letter sequence with HTTP 200 — indistinguishable, at the protocol level, from a real sign. Publishing such a clip as a word sign would teach learners something actively wrong. Visual

review alone proved unreliable (a reviewer who does not know Jordanian Sign Language cannot always tell an unfamiliar sign from a spelled sequence), so an automated classifier was built:

1. For a candidate word, fetch the word itself AND the doubled word (the word concatenated with itself) — the doubled form is guaranteed to be out-of-vocabulary, so its returned animation is by construction the cloud's own spelling of the word, processed by the identical pipeline.
2. Reduce both animations to per-frame handshape descriptors: the 21 right-hand landmarks, wrist-relative and scale-normalized, so global position and size differences cancel.
3. Compare with start-anchored dynamic time warping (DTW), which absorbs the timing differences the stitching pipeline introduces. If the word's animation matches the spelling's prefix, it IS the spelling fallback; scores separate cleanly (spelled < 0.9 , real ≥ 2.9).

Every accepted clip was additionally reviewed frame-by-frame as a rendered filmstrip. The sweep of ~170 candidate Arabic words produced two further discoveries: the dictionary keys many entries under proper orthography only (اَل is a real sign while ا comes back spelled — the direct motivation for the fold-everywhere lookup of Section 13.6), and words containing ð crash the API with HTTP 500 when out-of-vocabulary (its spelling lexicon lacks that letter), which turns the status code into a truth oracle for that word class. The final Arabic vocabulary — 25 verified signs — is exactly the set that passed all checks; everything else fingerspells honestly with the dashed  treatment.

13.9 Performance and Verification Engineering

Purpose: To record the decisions that keep the section responsive and the regression-testing approach.

- **One stitched file per playback.** All multi-sign animation is assembled server-side and cached (lru_cache), so the client performs exactly one media fetch per Play press regardless of sentence length — the fix for the original choppy, per-clip playback.
- **A single persistent viewer element.** Each tool keeps one <pose-viewer> mounted across word and tab switches (hidden when a fallback renderer is active), so switching words never re-initializes the rendering component; idle previews load the first frame only.
- **Guarded interaction states.** Word buttons, letter grids and inputs are disabled during playback; playback promises resolve on the component's ended\$ event, a cancellation poll, or a hard timeout, so the UI can never wedge in a playing state.
- **End-to-end browser verification.** Every change to the section was verified in a scripted headless Chrome session (Playwright): log in as a QA user, drive the real UI — switch language, play specific words, type sentences, assert chip classifications — and require zero browser console errors. Sign-content changes were additionally verified by rendering filmstrip sheets of the actual pose data and reviewing them visually.

13.10 Summary

The Education section pairs three tools — a categorized Sign Viewer with integrated fingerspelling, a free-text Sentence Builder, and a webcam Practice mode — across two sign languages behind one bilingual architecture. Signs are motion-capture skeletons of real signers (ASL from sign.mt's ase dictionary; Jordanian Sign Language for Arabic), rendered by the pose-viewer web component and stitched server-side into single smooth animations with timing metadata driving synchronized UI highlights. Arabic support required orthography-folded lookup on both client and server, right-to-left interface treatment, and a rigorous DTW-based verification methodology to reject the dictionary API's silent fingerspelling fallbacks — ensuring that every word the page presents as a sign is genuinely that sign, and that everything else is honestly labelled as fingerspelling. Practice mode closes the loop by checking the learner's own signing against the same recognition models that power live translation, offering only targets those models can actually recognize.